

UNIVERSITE PAUL-VALERY MONTPELLIER 3

Département MIASHS - UFR VI

Rapport

Science des Données 4

Prédiction de la Gravité des Accidents de la Route par
Apprentissage Automatique

présenté par:

Mélissa BOCCACCIO (22400372)

Maxime BOUTEYRE (22313124)

Camille SEVEYRAT (22303443)

réalisé sous la direction de :

Michel-Picque Théodore

Patel Namrata

Bringay Sandra

1. Introduction	3
2. Données	3
2.1. Source des données	3
2.2. Structure des données	3
3. Construction du jeu d'étude	5
3.1. Analyse exploratoire et visualisation	5
3.2. Sélection des variables	8
3.3. Construction du jeu de données	9
4. Protocole de préparation des données	10
4.1. Chargement et préparation des données	10
4.2. Entraînement des modèles	11
4.3. Validation chronologique	11
5. Modèles de machine learning	12
5.1. Méthodologie d'évaluation des modèles	12
5.2. Modèles	13
5.2.1. Logistic Regression	13
5.2.2. Decision Tree	15
5.2.3. Random Forest	17
5.2.4. K-Nearest Neighbors	19
5.2.5. Gradient Boosting	21
6. Optuna	23
6.1. Logistic Regression	24
6.2. Random Forest	26
6.3. Gradient Boosting	28
6.4. Conclusion	30
7. Analyse des variables importantes (Feature Importance)	30
7.1. Logistic Regression	31
7.2. Random Forest	32
7.3. Gradient Boosting	33
8. Sélection du meilleur modèle	34
9. Création du site web	36
9.1. Architecture du site	36
9.2. Préparation des modèles	36
9.3. Flask : moteur applicatif du site	37
9.3. Render	37

1. Introduction

Ce projet porte sur l'analyse des accidents de la route et la prédiction de leur gravité à l'aide de méthodes d'apprentissage automatique. En effet l'analyse des accidents de la route constitue un outil essentiel pour orienter les actions de sécurité, les choix d'aménagement urbain et les campagnes de sensibilisation visant à réduire leur fréquence et leur gravité. Dans cette perspective, l'étude de modèles de machine learning dotés de paramètres les plus performants pour prédire la gravité des accidents représente un enjeu majeur, elle offre un véritable atout pour la santé publique et pour l'élaboration de politiques efficaces de prévention routière.

Dans le cadre d'un travail universitaire nous nous sommes interrogés sur la problématique suivante : **"Comment être capable de prédire de manière fiable la gravité d'un accident de la route à partir de modèles de machine learning optimisés, afin d'améliorer la prévention et l'aide à la décision en matière de sécurité routière ?"**

Pour cela, plusieurs étapes ont été réalisées dans l'objectif de créer un site web capable de prédire la gravité d'un accident en fonction de différentes variables. Nous avons d'abord sélectionné une base de données complète et conséquente, puis effectué une analyse exploratoire accompagnée de visualisations afin d'identifier et de conserver les variables les plus significatives.

Par la suite, nous avons procédé à l'entraînement, l'évaluation et la comparaison de différents modèles d'apprentissage supervisé avant de développer une application web, intégrant un formulaire interactif permettant d'effectuer des prédictions grâce aux modèles de machine learning préalablement sélectionnés et optimisés.

L'ensemble du code source, des notebooks, des visualisations et des fichiers nécessaires à la reproduction du projet est disponible sur notre dépôt GitHub : [Mkoznzi/Science_des_donnees_4](https://github.com/Mkoznzi/Science_des_donnees_4)

2. Données

2.1. Source des données

Les données utilisées dans cette étude proviennent du jeu de données public disponible sur la plateforme officielle data.gouv.fr, intitulé « Bases de données annuelles des accidents corporels de la circulation routière ».

Parmi les années disponibles, nous avons sélectionné les données allant de **2014 à 2020**, en excluant l'année 2019 en raison de la présence d'un grand nombre de valeurs aberrantes (nombres composés de décimales importantes) et de valeurs manquantes susceptibles de biaiser l'analyse. Ce choix permet de constituer un jeu de données plus riche et représentatif, mais aussi d'évaluer la capacité des modèles à généraliser d'une année à l'autre.

2.2. Structure des données

Pour chaque année, les données sont réparties en quatre tables relationnelles :

- caractéristiques : informations générales sur l'accident (date, luminosité, localisation...)
- lieux : informations sur le lieu de l'accident

- *véhicules* : informations sur les véhicules impliqués
- *usagers* : informations individuelles sur les usagers

Ainsi, pour une année donnée, on dispose par exemple des tables : *caracteristiques_2014*, *lieux_2014*, *vehicules_2014*, *usagers_2014*. Le même schéma se répète pour chaque année étudiée.

Notre variable cible, *grav* dans la tables *usagers*, se constitue de la manière suivante :

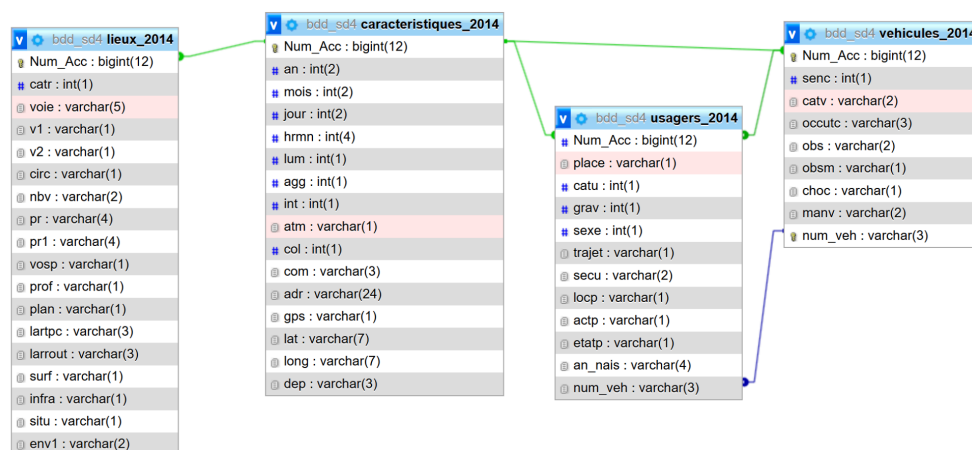
grav : Gravité de blessure de l'usager, les usagers accidentés sont classés en trois catégories de victimes plus les indemnes

- Valeurs
 - 1 – Indemne
 - 2 – Tué
 - 3 – Blessé hospitalisé
 - 4 – Blessé léger

Pour chaque année, nous avons importé l'ensemble des tables dans SQL, puis relié les tables d'une même année à l'aide de la clé primaire *Num_Acc*, issue de la table *caractéristiques*. On a également défini des clés primaire/étrangère cohérentes basées sur (*Num_Acc*, *num_veh*) pour garantir que chaque usager soit relié au bon véhicule dans le bon accident. Cela permettra de rendre la jointure fiable, sans doublons, et beaucoup plus performante lors de la création du jeu d'études, que nous allons voir par la suite.

Cependant, les types de données n'étaient pas homogènes d'une année à l'autre : certains champs *Num_Acc* étaient au format *int(2)* tandis que la majorité était en *bigint(12)*. Nous avons donc harmonisé ce champ en *bigint(12)* afin de garantir la compatibilité des données et permettre la création correcte des relations entre les tables.

La figure suivante présente un exemple des tables et des relations entre elles pour l'année 2014.



Nous n'avons volontairement effectué aucun nettoyage des bases de données. En effet, l'objectif étant d'identifier les variables ayant un impact sur la gravité des accidents (*grav* dans la table *usagers*), il était nécessaire de conserver l'intégralité des variables disponibles. Dans cette continuité, nous avons réalisé des analyses exploratoires et des visualisations afin d'identifier les variables potentiellement significatives dans l'objectif de prédire la gravité des accidents.

3. Construction du jeu d'étude

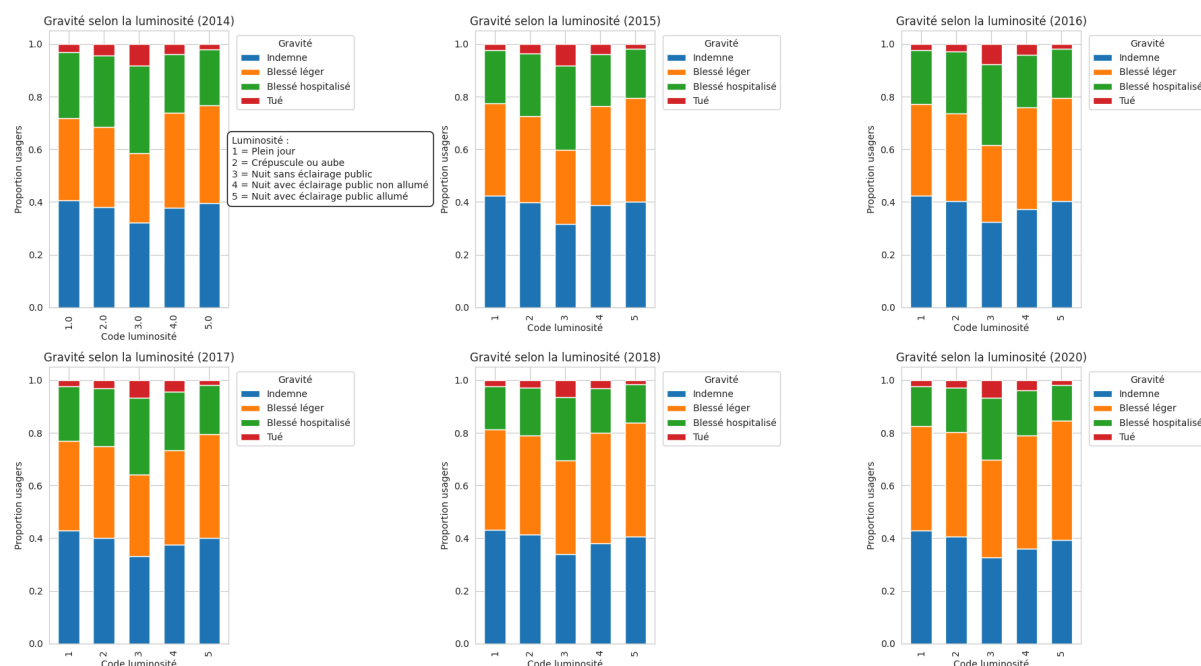
3.1. Analyse exploratoire et visualisation

L'objectif principal est de comprendre la structure des données, identifier les relations entre les variables, et déterminer quelles informations sont les plus pertinentes pour prédire la gravité des accidents.

Cette partie se concentre donc sur l'exploration des variables et leurs interactions, avec un accent particulier sur les facteurs pouvant influencer la gravité des accidents.

Nous avons commencé avec des visualisations assez générales pour avoir un aperçu des tables, comme l'évolution par années des accidents, ou leur répartition spatiale en France etc. Ensuite, nous avons pu commencer à cibler les variables et visualiser leur lien avec la gravité de blessure de l'usager.

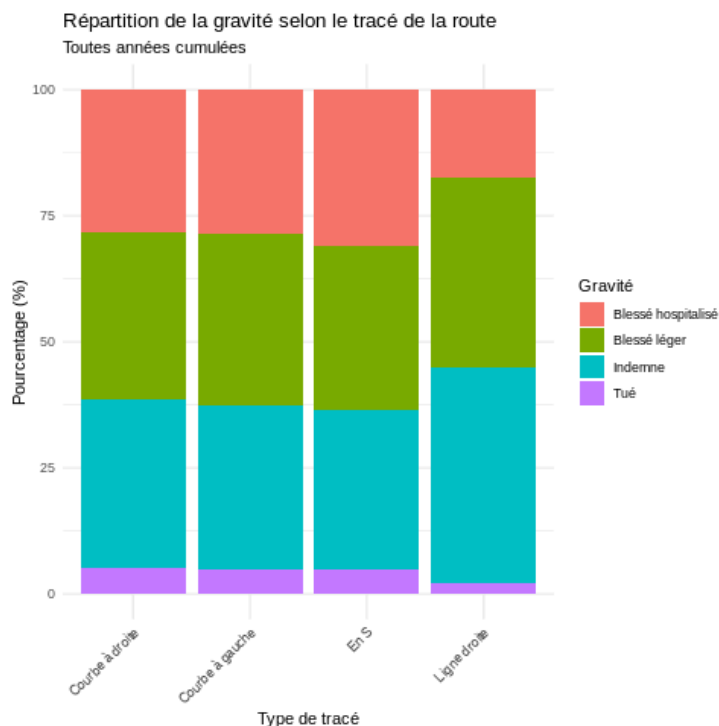
Par exemple, ici, nous avons analysé comment la gravité des accidents varie selon les conditions de luminosité. L'objectif est de voir si certaines conditions (plein jour, crépuscule, éclairages...) sont associées à un risque plus élevé de blessés graves ou de tués.



Graphiques en barres : proportion du niveau de gravité des blessures de l'usager selon les conditions de luminosité (2014–2020, hors 2019)

Les tendances sont relativement stables d'une année sur l'autre. On observe que les accidents survenant la nuit sans éclairage public présentent une proportion plus élevée de blessés hospitalisés et de tués, ce qui indique que ces conditions sont particulièrement à risque. La variable *lum* est donc pertinente pour la construction du jeu d'étude.

Des variables comme le type de tracé de la route, montrent elles aussi une influence claire sur la gravité des accidents.

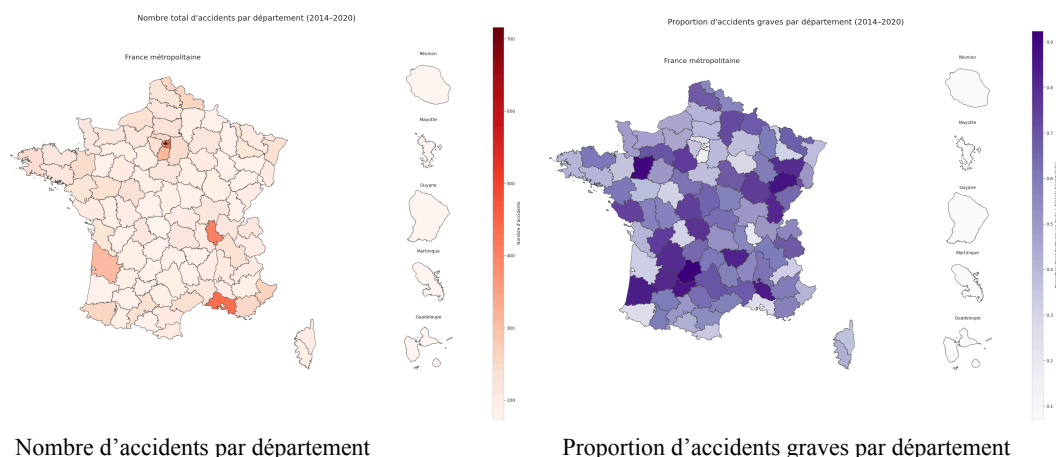


Graphique en barres : proportion du niveau de gravité des blessures de l'utilisateur selon le type de tracé de la route (années cumulées)

En effet, le tracé en ligne droite se distingue nettement. Il présente beaucoup moins de tués et de blessés hospitalisés que les autres types de tracé, avec une majorité d'indemnes. Les tracés en courbe (à gauche, à droite ou en S), où la visibilité est généralement réduite et la trajectoire plus complexe, semblent associés à des accidents plus graves, avec une proportion plus élevée de tués et de blessés hospitalisés par rapport aux lignes droites. Cette observation suggère que la configuration de la route joue un rôle dans la gravité des accidents, même si le nombre absolu d'accidents reste largement dominé par les lignes droites en raison de leur fréquence.

Lorsqu'on analyse la relation entre la gravité des accidents et différentes variables explicatives, il ne suffit pas d'observer uniquement leur lien apparent avec la gravité. Il est tout aussi essentiel d'examiner la manière dont ces variables se répartissent dans l'ensemble des données. En effet, pour interpréter correctement l'influence potentielle d'une variable, il faut connaître le poids relatif qu'elle représente dans le jeu de données. Une modalité très fréquente n'aura pas le même impact statistique qu'une modalité rare. Autrement dit, avant de conclure à un effet réel d'une variable sur la gravité, il est nécessaire d'analyser simultanément sa distribution et sa fréquence, afin d'éviter des interprétations biaisées et de replacer les résultats dans leur contexte réel.

Dans cette logique, les départements constituent un exemple pertinent. Ils permettent d'examiner comment les accidents se répartissent sur le territoire et d'évaluer si certaines zones présentent une gravité plus marquée que d'autres. Les cartes suivantes présentent successivement ces deux dimensions.



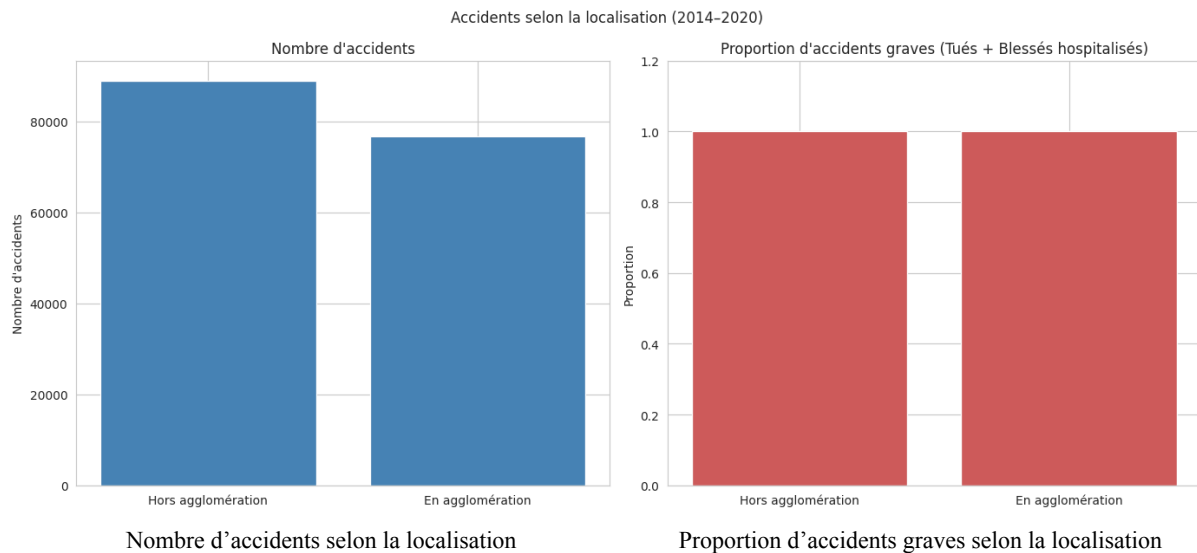
Les départements enregistrant le plus grand nombre d'accidents (comme Paris, les Bouches-du-Rhône ou le Rhône) apparaissent très clairement sur la carte. Cette forte concentration s'explique en grande partie par la densité de population et l'intensité du trafic, mais elle ne s'accompagne pas d'une gravité plus élevée : la proportion d'accidents graves y reste relativement faible, probablement en raison de vitesses moyennes plus faibles ou de conditions de circulation particulières. À l'inverse, plusieurs départements moins densément peuplés, tels que la Mayenne, le Vaucluse, les Landes, le Lot ou la Haute-Saône, présentent une proportion plus importante d'accidents graves. Ces contrastes montrent que la dimension territoriale apporte une information pertinente pour comprendre les variations de gravité, même si son influence reste modérée. Pour cette raison, nous avons choisi de conserver cette variable dans notre jeu de données final.

Même si certaines variables ne présentent pas de corrélation statistiquement significative avec la gravité lorsqu'elles sont examinées isolément, elles peuvent malgré tout apporter une information utile au modèle. Leur contribution peut être faible, mais elle peut néanmoins :

- réduire légèrement l'incertitude globale,
- aider le modèle à mieux distinguer les classes,
- améliorer la stabilité ou la cohérence des prédictions.

Autrement dit, une variable peu explicative prise seule peut tout de même renforcer la performance et la robustesse du modèle lorsqu'elle est intégrée dans un ensemble plus large de caractéristiques.

À l'inverse, certaines variables ne montrent pas d'effet notable sur la gravité. C'est le cas de la localisation de l'accident (en agglomération ou hors agglomération) : comme l'illustre le graphique ci-dessous, la proportion d'accidents graves est quasiment identique entre les deux contextes, même si le nombre total d'accidents diffère. Autrement dit, cette variable influence surtout la fréquence des accidents, mais pas leur gravité.



L'analyse exploratoire menée nous a permis d'identifier les variables présentant un lien potentiel avec la gravité des accidents, qu'il s'agisse de facteurs environnementaux (luminosité, tracé de la route), spatiaux (départements) ou liés aux usagers et aux véhicules. Cette étape a également montré que certaines variables, même faiblement corrélées lorsqu'elles sont observées isolément, peuvent malgré tout apporter une information utile dans un contexte prédictif, notamment en améliorant la stabilité du modèle.

Les exemples présentés dans cette section ne constituent qu'un aperçu : cette démarche d'exploration et d'évaluation a été appliquée de manière systématique à l'ensemble des variables issues des quatre tables (lieux, caractéristiques, véhicules et usagers) afin de sélectionner un ensemble de caractéristiques cohérent et pertinent pour l'apprentissage automatique et la prédiction de la gravité.

3.2. Sélection des variables

Sur la base des observations précédentes, nous avons décidé de sélectionner les variables les plus pertinentes pour la construction du jeu d'étude.

Certaines variables techniques, comme *Num_Acc* et *num_veh*, ont été conservées uniquement pour permettre la reconstruction et la fusion des différentes tables, mais elles n'ont pas été utilisées comme variables explicatives dans le modèle.

Les variables retenues pour l'analyse et la prédiction sont les suivantes :

- *lum* : conditions de luminosité au moment de l'accident
- *col* : type de collision
- *mois* : mois de survenue de l'accident
- *hrmn* : heure et minute de l'accident
- *dep* : département
- *plan* : tracé de la route
- *num_veh* : identifiant du véhicule
- *obs* : obstacle fixe heurté

- *catv* : catégorie du véhicule
- *catu* : catégorie d'utilisateur
- *sexe* : sexe de l'utilisateur
- *an_nais* : année de naissance de l'utilisateur
- *grav* : niveau de gravité de l'accident pour l'utilisateur

3.3. Construction du jeu de données

Afin de constituer un jeu de données exploitable pour le machine learning, nous avons réalisé une jointure des différentes tables pour chaque année, en utilisant les identifiants d'accident (*Num_Acc*) et de véhicule (*num_veh*).

Cette opération permet de regrouper, pour chaque usager impliqué dans un accident, l'ensemble des informations pertinentes concernant l'accident, le véhicule et le contexte routier.

La requête SQL suivante illustre la construction de la table pour l'année 2014 :

```
CREATE TABLE `2014` AS
SELECT
  c.Num_Acc,
  c.lum,
  c.col,
  c.mois,
  c.hrmn,
  c.dep,
  l.plan,
  v.num_veh,
  v.obs,
  v.catv,
  u.catu,
  u.sexe,
  u.an_nais,
  u.grav

FROM caracteristiques_2014 AS c

LEFT JOIN lieux_2014 AS l
  ON c.Num_Acc = l.Num_Acc

LEFT JOIN usagers_2014 AS u
  ON c.Num_Acc = u.Num_Acc

LEFT JOIN vehicules_2014 AS v
  ON u.Num_Acc = v.Num_Acc
  AND u.num_veh = v.num_veh;
```

Notre jeu de données se compose donc de six tables distinctes, chacune correspondant à une année différente.

4. Protocole de préparation des données

4.1. Chargement et préparation des données

Nous avons commencé à créer trois fonctions utilitaires afin de simplifier le chargement, la préparation des données et le modèle d'entraînement.

La première fonction `charger_preparer_donnees(annee)` permet de charger, nettoyer, harmoniser et préparer les données de nos tables annuelles afin qu'elles soient directement utilisables par un modèle de machine learning.

Celle-ci consiste donc, dans un premier temps, à charger le fichier CSV avec le séparateur adéquat et à considérer les valeurs manquantes. Puis, elle permet la création de types cohérents (*Int64*, *string*...) pour chaque variable, ce qui est indispensable pour éviter les erreurs lors du traitement des données. Elle harmonise ensuite la notation des départements, car les tables de 2014 à 2018 utilisent un codage différent de celui de 2020. Dans ces anciens fichiers, la Corse était identifiée par les codes *201* et *202*, que la fonction convertit désormais en *2A* (Corse-du-Sud) et *2B* (Haute-Corse). De plus, les départements métropolitains y étaient notés avec un zéro final superflu (par exemple *130* au lieu de *13*). La fonction supprime donc ce zéro et aligne l'ensemble des codes sur la notation standard utilisée dans la table de 2020.

Une fois les données nettoyées et harmonisées, la fonction construit la variable cible "*gravite*". Les quatre modalités initiales sont regroupées en deux classes :

- **Non grave** : regroupe les usagers indemnes (1) et ceux légèrement blessés (4).
- **Grave** : regroupe les usagers tués (2) et ceux hospitalisés (3).

Cette transformation binaire permet de formuler le problème comme une prédiction de la gravité majeure d'un accident, ce qui est directement utile pour l'identification des situations à risque et la mise en place de mesures de prévention. Cela permet de passer d'un problème multi-classes à une classification binaire, plus simple et plus stable.

Ce regroupement permet de reformuler le problème en une classification binaire, centrée sur la prédiction de la gravité majeure d'un accident. La fonction définit alors les variables explicatives *X* en supprimant les identifiants techniques (*Num_Acc*, *num_veh*) ainsi que les colonnes de gravité d'origine (*grav*, *gravite*), et la variable cible *y* comme la nouvelle variable binaire *gravite*. Les variables catégorielles sont ensuite transformées en variables indicatrices via un encodage one-hot (`pd.get_dummies`), de manière à rendre l'ensemble des caractéristiques utilisables par les modèles de machine learning.

Après avoir défini cette fonction, nous l'appliquons successivement à chacune des années disponibles (2014, 2015, 2016, 2017, 2018 et 2020). Comme les fichiers annuels ne contiennent pas exactement les mêmes variables — certaines colonnes apparaissant ou disparaissant selon les années — une étape supplémentaire est nécessaire pour garantir l'homogénéité du jeu de données final.

Dans un premier temps, nous parcourons l'ensemble des années afin de recenser toutes les colonnes présentes au moins une fois. Cette liste globale sert ensuite de référence pour recharger chaque année : lors de cette seconde passe, chaque tableau est réindexé de manière à utiliser exactement les mêmes colonnes. Si une variable est absente pour une année donnée, elle est ajoutée et remplie par défaut (valeur 0), ce qui permet d'obtenir une structure identique pour toutes les années.

Cette harmonisation garantit que l'ensemble des données annuelles peut être concaténé ou utilisé conjointement sans provoquer d'erreurs, et constitue ainsi un jeu d'étude multi-années cohérent, prêt pour l'entraînement et l'évaluation des modèles.

4.2. Entraînement des modèles

La seconde fonction `entr_eval_modele(pipeline, X, y)` a pour objectif d'entraîner un modèle de machine learning encapsulé dans un pipeline, puis d'évaluer ses performances. Elle commence par diviser les données en deux ensembles grâce à `train_test_split` :

- 80 % pour l'entraînement,
- 20 % pour le test, afin d'évaluer le modèle sur des données qu'il n'a jamais vues.

Le pipeline est ensuite entraîné sur les données d'entraînement avec la méthode `fit` puis utilisé pour générer des prédictions sur l'ensemble de test avec `predict`.

Plusieurs métriques d'évaluation sont calculées :

- accuracy, qui mesure la proportion de prédictions correctes,
- matrice de confusion, qui détaille les le nombre de prédictions correctes et incorrectes,
- classification report, qui fournit précision, rappel et F1-score.

Pour obtenir une évaluation plus robuste, la fonction applique également une validation croisée sur les données d'entraînement : `scores = cross_val_score(pipeline, X_train, y_train, cv=5)`.

Avec `cv=5`, les données sont découpées en cinq parties : le modèle est entraîné cinq fois, en utilisant à chaque itération quatre plis pour l'entraînement et un pli différent pour le test. Cette procédure permet d'estimer la performance moyenne du modèle et de réduire l'influence d'un découpage particulier des données.

La fonction retourne finalement le score moyen de cette validation croisée, qui constitue une estimation plus stable de la performance du modèle sur l'année considérée.

Cette fonction a pour objectif d'obtenir une première évaluation des modèles sur chaque année prise individuellement, avant de passer sur une approche plus réaliste fondée sur la validation chronologique.

4.3. Validation chronologique

Cette dernière fonction, `validation_chronologique(pipeline, data, annees)` réalise une validation croisée en respectant l'ordre temporel des données. L'idée est d'entraîner les modèles sur les années passées, puis de les tester sur une année future. Contrairement à la validation croisée classique qui mélange aléatoirement les données, cette méthode respecte la causalité : le modèle est toujours entraîné sur les années passées et testé sur une année future. Tandis qu'une validation croisée classique permet de choisir le meilleur modèle et de mesurer ses performances intrinsèques, la validation chronologique permet de vérifier que le modèle reste performant dans le temps. En effet, nos bases de données étant étendues sur six années différentes, il est essentiel d'évaluer les performances des modèles dans cette logique temporelle afin de s'assurer qu'ils conservent de bonnes performances.

Tout d'abord, les années sont triées dans l'ordre chronologique croissant pour garantir la cohérence de l'entraînement. La validation se déroule sous forme de boucle : pour chaque année à partir de la deuxième, la fonction définit un ensemble d'entraînement constitué de toutes les années antérieures, et un ensemble de test constitué uniquement de l'année courante.

Par exemple :

- Première itération : train = [2014] → test = 2015,
- Deuxième itération : train = [2014, 2015] → test = 2016,
- Troisième itération : train = [2014, 2015, 2016] → test = 2017.

Ainsi, plus on avance dans le temps, plus le modèle dispose d'années d'entraînement. Les données d'apprentissage sont obtenues en concaténant toutes les années passées en une seule base, tandis que les données de test correspondent directement à l'année cible.

Une fois les données d'entraînement et de test constituées, la fonction entraîne le pipeline sur l'ensemble d'entraînement, puis génère des prédictions sur l'année de test. Elle calcule ensuite l'accuracy, c'est-à-dire la proportion de prédictions correctes, et ajoute ce score à la liste des résultats. À chaque itération, elle affiche les années utilisées pour l'entraînement, l'année utilisée pour le test, ainsi que le score obtenu, ce qui permet de suivre l'évolution des performances du modèle au fil du temps.

Une fois la boucle terminée, la fonction calcule la moyenne et l'écart-type des scores obtenus sur l'ensemble des splits chronologiques. La moyenne fournit une estimation globale de la performance du modèle pour prédire des années futures, tandis que l'écart-type renseigne sur la stabilité du modèle d'une année à l'autre. Enfin, la fonction retourne la moyenne des scores, ce qui permet de comparer facilement plusieurs modèles ou pipelines selon leur capacité à généraliser dans un contexte temporel réaliste.

5. Modèles de machine learning

5.1. Méthodologie d'évaluation des modèles

Après avoir construit un jeu de données propre, harmonisé et exploitable, ainsi qu'un ensemble de fonctions dédiées au chargement, à la préparation et à l'évaluation des données, nous pouvons désormais appliquer différents modèles de machine learning.

L'objectif de cette section est de comparer plusieurs algorithmes, allant de modèles linéaires simples à des méthodes d'ensemble plus complexes, afin d'identifier ceux qui offrent les meilleures performances pour la prédiction de la gravité des accidents.

Pour garantir une démarche rigoureuse et reproductible, l'évaluation de chaque modèle suit exactement le même protocole :

1. Construction d'un pipeline

Chaque modèle est intégré dans un pipeline comprenant :

- *SimpleImputer* : imputation des valeurs manquantes (moyenne pour les variables numériques)

- *StandardScaler* : normalisation des variables lorsque le modèle y est sensible
- *Le modèle ainsi que ses paramètres*

2. Évaluation année par année

Pour chaque année du jeu de données, nous :

- Récupérons les données préparées (X, y),
- Entraînons et évaluons le modèle via la fonction `entr_eval_modele`,
- Collectons le score moyen de validation croisée.

Cette analyse permet d'évaluer la stabilité du modèle d'une année à l'autre et de détecter d'éventuelles variations dans les données.

3. Validation chronologique

Enfin, nous appliquons la fonction `validation_chronologique`, qui respecte l'ordre temporel des données. Cette étape permet d'évaluer la capacité du modèle à généraliser vers des années futures, ce qui correspond au scénario réel d'utilisation.

5.2. Modèles

Le code employé pour former et évaluer les divers modèles reste largement similaire d'un modèle à l'autre. La distinction se trouve uniquement dans le pipeline, qui varie selon le modèle sélectionné et leurs paramètres. Nous allons par conséquent présenter et analyser chaque pipeline en précisant le modèle employé ainsi que l'explication de ses paramètres, puis nous procéderons à l'analyse et à l'interprétation des performances.

5.2.1. Logistic Regression

A. Pipeline

```
pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])
```

La régression logistique est un modèle linéaire utilisé pour la classification binaire. Dans notre cas, elle permet d'estimer la probabilité qu'un accident soit grave en fonction des variables explicatives. Ce modèle servira de référence pour comparer les performances avec des modèles plus complexes.

Dans le pipeline associé, un *StandardScaler* est appliqué avant l'entraînement, car la régression logistique est sensible à l'échelle des variables : la normalisation permet d'assurer une convergence stable de l'algorithme d'optimisation et d'éviter qu'une variable à grande amplitude domine les autres.

Puis, lors de l'entraînement du modèle, le paramètre `max_iter=1000`, correspond au nombre maximal d'itérations autorisées pour l'algorithme d'optimisation. A chaque itération, le modèle s'améliore petit à petit en ajustant ses coefficients pour réduire l'erreur.

Le processus peut être résumé en quatre étapes :

1. Evaluation de l'erreur actuelle

L'algorithme calcule la valeur de la fonction de perte (la log-loss), qui mesure l'écart entre les probabilités prédites et les valeurs réelles.

2. Calcul du gradient

Le gradient est le vecteur qui donne la direction à suivre pour réduire l'erreur en indiquant comment modifier chaque coefficient dans l'objectif d'une amélioration du modèle.

3. Mise à jour des coefficients

Les paramètres du modèle sont ajustés progressivement dans la direction opposée au gradient, ce qui permet de réduire la perte étape après étape.

4. Vérification de la convergence

L'algorithme vérifie si les mises à jour deviennent suffisamment petites pour considérer que la solution est stable. Si ce n'est pas le cas, une nouvelle itération commence, jusqu'à atteindre la convergence ou la limite *max_iter*.

Dans notre cas, le jeu de données est volumineux et comporte de nombreuses variables après encodage, ce qui peut nécessiter un nombre élevé d'itérations pour atteindre une solution stable. L'utilisation de *max_iter=1000*, combinée à la normalisation via *StandardScaler*, garantit que l'algorithme dispose d'un nombre suffisant de mises à jour pour converger correctement.

B. Interprétation

1. Accuracy

Le score de l'accuracy tourne autour de 80% de manière très constante (de 79,5% en 2015 à 83,9% en 2020).

Cependant, cette métrique doit être interprétée avec prudence : notre jeu de données est fortement déséquilibré, avec environ 3 à 4 fois plus de données pour la classe 0 (qui représente les cas non graves) que pour la classe 1 (qui représente les cas graves, et donc ceux qui représentent la priorité de nos prédictions). Le modèle de régression logistique, au vu de nos résultats, semble performant pour prédire la classe majoritaire mais peine sur la classe 1.

2. Analyse des performances par classe

- **Classe 0 (Cas non graves) :** Les scores sont excellents.
 - Le Recall est de 0.93 - 0.95, ce qui signifie que le modèle identifie presque tous les individus de la classe 0.
- **Classe 1 (Cas graves) :** Les scores sont moins bons.
 - Le Recall est bas (entre 0.29 et 0.37). Cela signifie que le modèle ne détecte que 30 à 37 % des cas réels de la classe 1. Il manque donc environ 65 à 70 % des cas.
 - La précision est correcte (autour de 0.60 - 0.64). Lorsque le modèle prédit un cas grave, il a raison environ 6 fois sur 10.
 - Le F1-Score (la moyenne entre précision et recall) est assez faible pour cette classe (0.39 à 0.47), confirmant que le modèle n'est pas très performant pour capturer les "positifs".

3. La Validation Croisée Chronologique

La validation chronologique permet d'évaluer la capacité du modèle à généraliser d'une année à l'autre. Les résultats montrent une stabilité remarquable : le score moyen est de 0.7997 avec un écart-type très faible (0.017).

Cela indique que le modèle ne fait pas de sur-apprentissage. Il est très robuste et ses performances ne s'effondrent pas d'une année sur l'autre. Il a bien capturé des tendances de fond qui restent vraies entre 2014 et 2020.

3. Analyse de la Matrice de Confusion (Exemple 2014)

- 18 877 Vrais Négatifs : le modèle a correctement identifié la classe 0.
- 2 252 Vrais Positifs : il a correctement détecté la classe 1.
- 4 067 Faux Négatifs : il a prédit « non grave » alors qu'il s'agissait d'un cas grave ; ce sont les cas graves manqués.
- 1 242 Faux Positifs : le modèle a prédit « grave » alors que l'accident n'était pas grave.

REEL 0	18 877	1 242
REEL 1	4 067	2 252
	PREDIT 0	PREDIT 1

Cette matrice montre un comportement typique de la régression logistique dans un contexte de classes déséquilibrées.

Les Faux Positifs restent modérés, ce qui signifie que le modèle ne sur-réagit pas en prédisant trop souvent la classe 1.

En revanche, les Faux Négatifs sont élevés, ce qui confirme que la régression logistique a du mal à détecter les accidents graves.

4. Résumé

Le modèle de Régression Logistique semble être un bon point de départ, mais il est trop imprécis pour les cas graves. Cependant, en raison du fort déséquilibre des classes, il a tendance à favoriser la classe majoritaire. Cela se traduit par de bonnes performances globales, mais par une détection insuffisante des accidents graves, qui restent largement sous-prédits.

5.2.2. Decision Tree

A. Pipeline

```
pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("model", DecisionTreeClassifier(random_state=42))
])
```

L'arbre de décision est un modèle non linéaire qui segmente les données en appliquant une succession de règles du type : "Si la variable $X \leq 3.2$ alors classe A, sinon classe B". Il segmente l'espace des données en zones homogènes.

À chaque étape, le modèle choisit la variable et le seuil qui permettent de séparer au mieux les observations selon la cible. Cette approche permet de capturer des relations complexes et des interactions entre variables, contrairement aux modèles linéaires comme la régression logistique.

Dans notre pipeline, l'arbre de décision n'a pas besoin de normalisation préalable : il repose uniquement sur des comparaisons de seuils, et l'échelle des variables n'influence pas la construction de l'arbre.

L'entraînement du modèle repose sur un processus itératif de construction de l'arbre. À chaque étape, l'algorithme :

1. **Évalue la qualité d'une séparation possible**

Il mesure à quel point un seuil sur une variable permet de rendre les groupes plus homogènes.

2. **Sélectionne la meilleure règle de décision**

L'algorithme choisit la variable et le seuil qui maximisent la séparation entre les classes.

3. **Crée deux nouveaux nœuds**

Les données sont divisées en deux sous-ensembles, chacun devenant un nœud de l'arbre.

4. **Répète le processus récursivement**

Chaque nœud est à son tour séparé tant que cela améliore la pureté des groupes ou jusqu'à atteindre une limite (profondeur maximale, nombre minimal d'échantillons, etc.).

Ce mécanisme permet au modèle d'apprendre des règles très spécifiques aux données. Cependant, un arbre trop profond peut mémoriser les données d'entraînement et perdre sa capacité à généraliser : c'est le surapprentissage. Le contrôle de la profondeur ou du nombre minimal d'échantillons par feuille est donc essentiel pour obtenir un modèle robuste.

Enfin, le paramètre *random_state* fixe l'aléatoire dans le modèle. Cela garantit que l'arbre est reproductible et les performances obtenues restent identiques. Cette pratique est essentielle car elle assure la stabilité, la traçabilité et la comparabilité des expériences.

B. Interprétation

1. Accuracy

Le modèle affiche une précision globale située entre 74% et 79%.

- Tendance : Le score est très stable entre 2014 et 2017 (~74-75%), puis il grimpe légèrement pour atteindre 79,1% en 2020.
- Interprétation : Dans l'ensemble, le modèle répond correctement dans environ 3 cas sur 4. C'est un score solide pour un modèle simple comme un arbre de décision.

2. Analyse des performances par classe

Contrairement au modèle de régression logistique, l'arbre de décision essaie de capturer la classe minoritaire et d'augmenter la précision de sa prédiction de ces cas-là.

- **Classe 0 (Cas non graves) :** Les scores sont très bons et équilibrés (Précision et Recall autour de 0.84). Le modèle reconnaît bien les cas "normaux".
- **Classe 1 (Cas graves) :**
 - Le Recall est d'environ 46-48%. Cela signifie que le modèle parvient à détecter presque la moitié des cas réels de la classe 1.
 - La Précision est également autour de 46-48%. Cela veut dire que lorsqu'il prédit une classe 1, il a raison environ une fois sur deux.

3. Analyse de la Matrice de Confusion (Exemple 2017)

Pour mieux comprendre, regardons les chiffres concrets de l'année 2017 :

REEL 0	17 275	3 472
REEL 1	3 346	3 112
	PREDIT 0	PREDIT 1

On observe donc que le modèle ne commet pas uniquement les erreurs dans un sens : il confond fréquemment les deux classes, ce qui traduit une difficulté à séparer correctement les accidents graves des non graves.

4. Validation Croisée Chronologique

Le modèle montre une bonne stabilité : son score moyen global est de 0.757 avec un écart-type de 0.017 (soit seulement 1,7% de variation).

Les résultats sont extrêmement cohérents d'une année à l'autre et entre les différents "plis" (folds) de la validation croisée. Cela prouve que le modèle n'est pas victime de sur-apprentissage. Il a découvert des règles logiques qui restent valables sur 6 ans de données.

5. Résumé

C'est un modèle équilibré et proactif. Il est capable de détecter sans se contenter de prédire la classe majoritaire en prédisant assez correctement la classe 1. C'est également un modèle robuste car les règles qu'il utilise pour réaliser sa prédiction sont stables dans le temps (écart-type très faible).

Cependant, sur la classe 1, il a un taux de réussite qui reste faible, d'environ 50%. C'est utile pour dégrossir un problème, mais cela génère un nombre de fausses alertes (Faux Positifs), quasiment égal au nombre de bonnes détections.

5.2.3. Random Forest

A. Pipeline

```
pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("model", RandomForestClassifier(
        n_estimators=50,
```

```
n_jobs=-1,  
    random_state=42  
))  
])
```

La forêt aléatoire (Random Forest) est un modèle d'ensemble basé sur une collection d'arbres de décision. L'idée centrale est de réduire l'instabilité propre aux arbres individuels en entraînant un grand nombre d'arbres sur des sous-échantillons différents des données, puis en agrégeant leurs prédictions. Cette approche permet d'obtenir un modèle plus robuste, moins sensible au bruit, et généralement plus performant qu'un arbre unique.

Dans notre pipeline, chaque arbre est construit à partir d'un échantillon tiré aléatoirement avec remise (méthode du bootstrap), et à chaque nœud, seule une fraction aléatoire des variables est testée pour déterminer la meilleure séparation. Cette double source d'aléa (échantillons et variables) permet de diversifier les arbres et d'éviter qu'ils apprennent tous les mêmes règles, ce qui améliore la généralisation du modèle.

L'entraînement d'une forêt suit un processus structuré :

1. **Génération d'un sous-échantillon bootstrap**

Pour chaque arbre, un échantillon aléatoire des données est tiré avec remise, ce qui crée des variations entre les arbres.

2. **Construction d'un arbre de décision**

L'arbre est entraîné sur ce sous-échantillon, en testant à chaque nœud seulement un sous-ensemble aléatoire de variables, ce qui renforce la diversité des arbres.

3. **Répétition du processus pour chaque arbre**

Le modèle construit ainsi un ensemble d'arbres indépendants, chacun capturant des aspects différents des données.

4. **Agrégation des prédictions**

Pour la classification, la prédiction finale est obtenue par vote majoritaire entre les arbres, ce qui stabilise le modèle et réduit le risque de surapprentissage.

Le paramètre `n_estimators=50` indique que la forêt est composée de 50 arbres. Un nombre plus élevé d'arbres tend à améliorer la stabilité et la performance du modèle, au prix d'un temps de calcul plus important. Le paramètre `n_jobs=-1` permet d'exploiter l'ensemble des cœurs du processeur, ce qui permet d'accélérer fortement l'entraînement. Enfin, `random_state=42` fixe la graine aléatoire utilisée lors de la construction des arbres, garantissant ainsi la reproductibilité des résultats.

B. Interprétation des résultats

1. Accuracy

Le score d'accuracy est assez stable. Il démarre à 80,4 % en 2014 et grimpe jusqu'à 83,6 % en 2020. Cela signifie que ce modèle est très performant pour donner la bonne réponse dans la grande majorité des cas. On observe aussi une amélioration naturelle du score avec le temps (2018 et 2020 sont les meilleures années), ce qui peut indiquer que les données récentes sont un peu plus faciles à classer pour lui ou que les motifs deviennent plus clairs.

2. Analyse des performances par classe

- **Classe 0 (Cas non graves) :** Le modèle est presque parfait. Avec un Recall de 0.92 à 0.95, il identifie presque systématiquement les accidents non graves. Autrement dit, lorsqu'un individu appartient réellement à la classe 0, le modèle le classe correctement dans la grande majorité des cas.
- **Classe 1 (Cas graves) :**
 - Précision (0.60 - 0.65) : c'est l'aspect le plus solide du modèle pour cette classe. Lorsqu'il prédit un accident grave, il a raison dans environ 60 à 65 % des cas, ce qui traduit un niveau de confiance raisonnable dans ses prédictions positives.
 - Recall (0.32 - 0.42) : c'est la principale limite. Le modèle ne détecte que 30 à 40 % des accidents réellement graves. Il reste donc très sélectif et laisse échapper une proportion importante de cas graves.

3. Analyse de la Matrice de Confusion (Exemple 2016)

Regardons les chiffres pour l'année 2016 pour illustrer ce comportement :

REEL 0	18 794	1 478
REEL 1	3750	2 663
	PREDIT 0	PREDIT 1

Le modèle génère relativement peu de faux positifs par rapport au volume total de données, mais il laisse échapper une proportion importante de cas graves (Faux Négatifs).

4. Validation Croisée Chronologique

Les résultats de la validation croisée sont d'une bonne régularité. Le score moyen global obtenu est de 0.8105 avec un écart-type de 0.0139 (soit seulement 1,3 % de variation). C'est la signature d'un modèle extrêmement fiable. Cette régularité indique que le modèle ne présente pas de sur-apprentissage : il conserve des performances similaires lorsqu'il est entraîné sur les années précédentes et testé sur une année future. Le Random Forest apprend donc des règles robustes, applicables de manière cohérente sur l'ensemble de la période 2014–2020.

5. Résumé

Le modèle Random Forest est un modèle équilibré, fiable et robuste.

Il présente une bonne précision sur la classe 1 : lorsqu'il prédit un accident grave, il a tendance à avoir raison. De plus, ses résultats ne varient que très peu d'une année sur l'autre. Pour autant, sa prédiction des cas graves reste encore faible : il en rate plus de la moitié (Recall < 0.50).

5.2.4. K-Nearest Neighbors

A. Pipeline

```
pipeline = Pipeline([
```

```

("imputer", SimpleImputer(strategy="mean")),
("model", KNeighborsClassifier(
    n_neighbors=5,
    algorithm="kd_tree",
    n_jobs=-1
))
])

```

Le pipeline utilisé combine une étape d'imputation des valeurs manquantes et un modèle KNeighborsClassifier.

Le modèle utilisé est un K-Nearest Neighbors (KNN), un algorithme de classification qui prédit la catégorie d'une donnée en se basant sur les 5 voisins les plus proches dans l'espace des caractéristiques. Lorsqu'une nouvelle donnée doit être prédite, le modèle agit en trois étapes précises :

1. Calcul de distance : Il mesure la distance (généralement euclidienne) entre cette nouvelle donnée et toutes les données du jeu d'entraînement.
2. Sélection des voisins : Il identifie les 5 données les plus proches
3. Il examine les classes de ces 5 voisins. La classe prédite pour la nouvelle donnée est celle qui apparaît le plus souvent parmi eux (par exemple, si 3 voisins sont de classe A et 2 de classe B, la prédiction est A).

Le paramètre `n_neighbors=5` fixe ce nombre de voisins, tandis que `algorithm="kd_tree"` optimise les calculs pour les données de faible dimension en structurant l'espace sous forme d'arbre, et `n_jobs=-1` autorise l'utilisation de tous les cœurs du processeur pour accélérer le processus.

B. Interprétation des résultats

1. L'Accuracy

L'accuracy globale du modèle se situe entre 73 % et 80 %. À première vue, on pourrait croire que le modèle est correct. En 2020, il atteint même 80 %. Cependant ce score est élevé uniquement parce que le modèle prédit presque tout le monde en Classe 0. Comme la classe 0 est ultra-majoritaire devant la classe 1, il obtient une bonne note globale alors qu'il passe à côté de l'essentiel.

2. Analyse des performances par classe

Le modèle KNN semble incapable de saisir les caractéristiques de la classe 1. Le Recall, très faible (entre 0.09 et 0.17), indique qu'il ne détecte que 9 % à 17 % des accidents graves. Autrement dit, il en manque entre 83 % et 91 %, ce qui le rend peu adapté à la détection des cas critiques.

La **Précision**, également limitée (entre **0.33 et 0.41**), montre que même lorsqu'il prédit un accident grave, il se trompe plus d'une fois sur deux.

Le **F1-score**, autour de **0.14**, confirme que le modèle ne parvient pas à apprendre des règles discriminantes pour identifier efficacement la classe 1.

Ces résultats traduisent une incapacité du KNN à distinguer les accidents graves des non graves dans notre contexte.

3. Analyse de la Matrice de Confusion (Exemple 2014)

Regardons les chiffres bruts pour comprendre l'ampleur du problème :

REEL 0	18 610	1 509
REEL 1	5 291	1 028
	PREDIT 0	PREDIT 1

Le modèle semble dominé par la forte présence de la classe 0 dans l'espace des données. La majorité des points environnants appartenant à la classe 0, le modèle a tendance à prédire systématiquement cette classe.

4. Validation Croisée Chronologique

Le score moyen global est de 0.754 avec un écart-type de 0.024. Le modèle est donc relativement stable d'une année à l'autre, mais cette stabilité reflète surtout sa difficulté persistante à détecter la classe 1. L'écart-type, légèrement plus élevé que pour les autres modèles, montre que le KNN est sensible aux variations de densité des données selon les années, ce qui est cohérent avec son fonctionnement basé sur la proximité locale.

5. Résumé

Le KNN est le modèle le moins performant de notre étude.

Il présente une **faible capacité de détection** des accidents graves ($\text{Recall} < 0.20$), une précision limitée et un F1-score très bas.

Dans un contexte où l'objectif principal est d'identifier la classe 1 avec le moins d'erreurs possibles, le KNN ne constitue pas une solution adaptée : son taux de faux négatifs est trop élevé pour être exploitable dans une application opérationnelle.

5.2.5. Gradient Boosting

A. Pipeline

```
pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("model", GradientBoostingClassifier())
])
```

Le Gradient Boosting Classifier est un modèle d'ensemble qui construit son prédicteur de manière progressive et séquentielle. Contrairement à la forêt aléatoire, où les arbres sont entraînés en parallèle, le Gradient Boosting ajoute les arbres les uns après les autres, chaque nouvel arbre ayant pour rôle de corriger les erreurs commises par les précédents. Cette stratégie permet d'obtenir un modèle particulièrement performant pour capturer des relations complexes dans les données.

Dans notre pipeline, le modèle est intégré après un *SimpleImputer*, ce qui garantit que les valeurs manquantes sont correctement traitées avant l'apprentissage. Le Gradient Boosting n'a pas besoin de normalisation préalable, car il repose sur des arbres de décision, insensibles à l'échelle des variables.

L'entraînement du Gradient Boosting suit un processus structuré :

1. Prédiction initiale

Le modèle commence par une estimation simple de la cible (par exemple une probabilité moyenne). Cette prédiction sert de point de départ.

2. Calcul des résidus

Le modèle calcule les erreurs commises sur chaque observation. Ces résidus représentent ce que le modèle doit apprendre à corriger.

3. Entraînement d'un nouvel arbre sur les résidus

Un arbre de décision peu profond est entraîné pour prédire ces résidus. Il apprend donc spécifiquement où le modèle précédent s'est trompé.

4. Mise à jour du modèle

Les prédictions de cet arbre sont ajoutées au modèle existant, pondérées par un taux d'apprentissage interne. Cela permet d'améliorer progressivement la prédiction globale.

5. Répétition du processus

Le modèle ajoute ainsi successivement de nouveaux arbres, chacun corrigeant les erreurs du modèle précédent, jusqu'à atteindre le nombre d'arbres prévu par les paramètres par défaut.

Dans notre cas, le modèle utilise les paramètres standards de *scikit-learn*, ce qui signifie que la profondeur des arbres, le nombre d'itérations et le taux d'apprentissage suivent les réglages recommandés par défaut. Ces valeurs sont généralement adaptées pour obtenir un bon compromis entre performance et stabilité, sans nécessiter de réglages manuels.

B. Interprétation des résultats

1. Accuracy

L'accuracy globale est élevée : elle oscille entre 79 % et 84 %. Comme pour les modèles précédents, on peut observer une amélioration sur les années 2018 et 2020. Le modèle est très performant pour donner une réponse correcte à l'échelle de l'ensemble de la base de données.

2. Analyse des performances par classe

Le Gradient Boosting adopte un comportement très différent de l'Arbre de Décision simple :

- **Classe 0 (Cas non graves) :** Les performances sont excellentes. Avec un Recall compris entre 0.96 et 0.98, le modèle identifie presque systématiquement les cas non graves : il commet très peu d'erreurs en classant un accident non grave comme grave.
- **Classe 1 (Cas graves) :**
 - **Précision (0.65 – 0.69) :** c'est l'un des points forts du modèle. Lorsqu'il prédit un accident grave, il a raison dans près de 7 cas sur 10, ce qui traduit une bonne fiabilité de ses prédictions positives.
 - **Recall (0.19 – 0.26) :** c'est la principale limite. Le modèle reste très sélectif et ne détecte qu'une faible proportion des accidents graves. Il privilégie la réduction des fausses alertes, au prix d'un nombre important de cas graves manqués.

3. Analyse de la Matrice de Confusion (Exemple 2014)

Regardons les chiffres pour bien visualiser :

REEL 0	19 378	741
REEL 1	4 782	1 537
	PREDIT 0	PREDIT 1

Le modèle a plutôt tendance à rester prudent dans ses prédictions de classe 1.

Les Faux Positifs sont relativement faibles par rapport au volume total, ce qui confirme que le modèle semble vouloir éviter de déclencher des alertes inutiles.

En revanche, les Faux Négatifs sont très nombreux, ce qui signifie que le modèle laisse passer une grande partie des cas graves.

4. Validation Croisée Chronologique

Le score moyen global est de 0.8029 et l'écart-type : 0.0171. Le Gradient Boosting est un des modèles les plus stables parmi les modèles testés. La validation croisée montre que le modèle ne divague pas. Il a appris des motifs très solides qui ne dépendent pas de la chance du découpage des données.

5. Résumé

C'est un modèle de haute qualité, mais il minimise les Faux Positifs (fausses alertes) au détriment des Faux Négatifs (cas manqués). Pour autant, il reste un modèle très fiable pour prédire les cas de la classe 1. Il est aussi très stable avec peu de variations d'une année sur l'autre. Cependant, il passe quand même à côté d'une partie importante des cas de la classe 1, ce qui pourrait être amélioré.

6. Optuna

Après avoir évalué l'ensemble des modèles, nous retenons les trois approches les plus performantes : Logistic Regression, Random Forest et Gradient Boosting.

A présent nous allons utiliser Optuna pour optimiser les performances de ces modèles. Optuna est une bibliothèque Python dédiée donc à l'optimisation des hyperparamètres pour le machine learning. Elle permet de trouver automatiquement la meilleure combinaison de réglages des modèles afin d'améliorer leurs performances.

Contrairement à une recherche aléatoire simple ou une grille de paramètres stricte, Optuna utilise des algorithmes d'optimisation avancés pour explorer l'espace des paramètres de manière intelligente. Le principe est le suivant :

1. **Suggestion** : Optuna propose une configuration de paramètres.
2. **Évaluation** : On entraîne le modèle avec ces paramètres et calcule une métrique de performance.
3. **Pruning** : Si le modèle semble avoir de mauvaises performances dès le début, Optuna peut arrêter l'entraînement prématurément pour gagner du temps et se concentrer sur des configurations plus prometteuses.
4. **Itération** : Le processus se répète jusqu'à trouver le meilleur jeu de paramètres.

L'objectif est d'identifier, pour chaque modèle, la configuration la plus efficace sur notre jeu de données.

6.1. Logistic Regression

```
import optuna

pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000))
])

def objective(trial):

    params = {
        "model__C": trial.suggest_float("C", 0.001, 10, log=True),
        "model__solver": trial.suggest_categorical("solver", ["lbfgs",
"liblinear"])
    }

    pipeline.set_params(**params)

    # Utiliser la validation chronologique comme métrique
    score = validation_chronologique(pipeline, data, annees)

    return score

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=20)

print("Best params :", study.best_params)
print("Best score :", study.best_value)
```

Ce code a donc pour objectif d'optimiser les hyperparamètres du modèle de régression logistique en utilisant la bibliothèque Optuna.

Comme pour les codes précédents l'utilisation d'un pipeline est nécessaire pour garantir que toutes les transformations sont appliquées de manière cohérente à chaque étape de l'optimisation. Une fonction *objective(trial)* est définie. Elle correspond à la fonction que l'algorithme d'optimisation cherche à maximiser.

À chaque appel de cette fonction, le code fait :

- Génération des hyperparamètres:
 - Optuna propose automatiquement une combinaison de paramètres à tester :
 - le paramètre de régularisation C, choisi dans l'intervalle [0.001,10] selon une échelle logarithmique. Il contrôle la régression logistique, plus C est grand, moins il y a de régularisation
 - le type de solveur (*solver*), qui est une méthode d'optimisation interne du modèle, sélectionné parmi deux options : *lbfgs* et *liblinear*.

- Mise à jour du modèle
Les paramètres proposés sont appliqués au pipeline grâce à `set_params`.
- Évaluation du modèle
Le modèle est évalué via une fonction `validation_chronologique`, qui met en œuvre une validation adaptée aux données temporelles (entraînement sur les années passées et test sur les années futures).
- Retour du score
La fonction retourne un score de performance, que l'algorithme cherche à maximiser.

Une étude est ensuite créée avec Optuna :

- `create_study(direction="maximize")` indique que l'objectif est de maximiser le score ;
- `study.optimize(objective, n_trials=20)` lance l'optimisation sur 20 essais.

À chaque essai, une nouvelle combinaison de paramètres est testée. Le modèle est entraîné puis évalué et le score obtenu est enregistré. Optuna utilise les résultats précédents pour explorer efficacement l'espace des paramètres.

Au total, 20 essais (trials) ont été effectués. Pour chaque essai, un couple de paramètres (C, solver) est testé. Le modèle est évalué via la validation chronologique (apprentissage sur les années passées, test sur l'année suivante) et un score moyen est calculé.

Les résultats montrent que les performances sont globalement proches d'un essai à l'autre, avec des scores compris entre 0.799 et 0.802.

À la fin du processus, deux éléments sont affichés :

- `best_params` : les hyperparamètres ayant donné les meilleures performances ;
- `best_value` : le score associé à cette meilleure configuration.

Ces paramètres peuvent ensuite être utilisés pour entraîner le modèle final.

Pour ce modèle les meilleurs hyperparamètres identifiés sont :

- `C = 0.00111`
- `solver = liblinear`

Le meilleur score moyen obtenu est de 0.80198 et indique que le modèle possède une bonne capacité de prédiction. L'écart-type associé au meilleur modèle est d'environ 0.016. Ce faible écart-type montre que les performances sont stables d'une année à l'autre. Les meilleurs résultats sont obtenus pour des valeurs de C très faibles (proches de 0.001). Cela signifie qu'une forte régularisation améliore les performances et que le modèle évite ainsi le surapprentissage. Le solveur `liblinear` apparaît légèrement meilleur que `lbfgs`, bien que les différences restent faibles.

La question est donc de savoir si cette optimisation était pertinente, avant optuna on avait un score moyen chronologique ≈ 0.7998 , après Optuna : ≈ 0.8020 . L'amélioration est faible mais réelle, de l'ordre de +0.002, ce qui reste dans la marge de stabilité du modèle.

Pour l'écart-type, avant Optuna : ≈ 0.0178 puis après Optuna : ≈ 0.0161 . Il y a une légère diminution, signe d'une stabilité encore renforcée. L'optimisation n'a pas transformé les performances du modèle, mais elle a permis de :

- légèrement améliorer le score moyen,

- réduire la variabilité,
- confirmer que le modèle fonctionne mieux avec une régularisation forte (C très petit),
- valider que le solver liblinear est adapté à ce type de données.

Ces résultats sont parfaitement cohérents avec les propriétés de la régression logistique :

- modèle linéaire : faible sensibilité aux hyperparamètres,
- performances limitées par la structure des données (déséquilibre, non-linéarités),
- optimum souvent atteint rapidement.

En d'autres termes, Optuna confirme que la régression logistique était déjà proche de son optimum naturel, et que ses limites viennent davantage de la nature du problème que de ses réglages.

6.2. Random Forest

```
import optuna

pipeline_rf = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("model", RandomForestClassifier(
        n_jobs=-1,
        random_state=42
    ))
])

def objective_rf(trial):
    params = {
        "model__n_estimators": trial.suggest_int("n_estimators", 50,
200),
        "model__max_depth": trial.suggest_int("max_depth", 5, 30),
        "model__min_samples_split": trial.suggest_int("min_samples_split", 2, 10),
        "model__min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 5),
        "model__max_features": trial.suggest_categorical("max_features", ["sqrt", "log2"])
    }

    pipeline_rf.set_params(**params)

    # Utilisation de la validation chronologique
    score = validation_chronologique(pipeline_rf, data, annees)

    return score

study_rf = optuna.create_study(direction="maximize")
study_rf.optimize(objective_rf, n_trials=20)

print("Best params RF :", study_rf.best_params)
print("Best score RF :", study_rf.best_value)
```

Pour ce modèle, la structure du code et l'objectif est le même que pour le modèle de régression logistique, à un détail près : les hyperparamètres sont différents, propres à Random Forest.

La fonction propose donc automatiquement une combinaison de paramètres à tester :

- *n_estimators* : Nombre d'arbres dans la forêt. Plus il y en a, plus le modèle est stable.
- *max_depth* : Profondeur maximale des arbres. Limite la complexité
- *min_samples_split* : Nombre minimum d'échantillons pour diviser un nœud
- *min_samples_leaf* : Nombre minimum d'échantillons dans une feuille
- *max_features* : Nombre de variables testées à chaque split (sqrt ou log2)

Comme pour la régression logistique, l'optimisation des hyperparamètres avec Optuna a été appliquée au Random Forest en utilisant la validation croisée chronologique, ce qui permet d'évaluer la capacité du modèle à généraliser sur des données futures.

Les meilleurs paramètres trouvés sont :

- *n_estimators* = 200
- *max_depth* = 30
- *min_samples_split* = 4
- *min_samples_leaf* = 2
- *max_features* = 'sqrt'

Score moyen obtenu : 0.8079

Ces paramètres ont ensuite été réévalués sur l'ensemble des splits chronologiques.

Globalement, l'optimisation apporte un léger gain de performance, plus marqué que pour la régression logistique, ce qui est cohérent avec la nature du Random Forest : un modèle plus flexible, plus sensible aux réglages, et capable de tirer parti d'une exploration plus fine de l'espace d'hyperparamètres.

Les scores obtenus sur chaque split (≈ 0.794 à 0.827) sont légèrement supérieurs à ceux obtenus avant optimisation, cela veut dire que le modèle gagne en cohérence et en stabilité, notamment sur les années récentes.

Pour les scores moyen chronologique sont, avant Optuna ≈ 0.7987 , puis après Optuna ≈ 0.8079 . Il y a donc une amélioration notable d'environ $+0.009$, ce qui est significatif pour un modèle déjà performant. Quant à l'écart-type, avant Optuna ≈ 0.0147 et après Optuna : ≈ 0.0157 , on y voit une légère augmentation, mais toujours dans une zone de stabilité acceptable.

Contrairement à la régression logistique, le Random Forest bénéficie réellement de l'optimisation: dans un premier temps dû à l'augmentation du nombre d'arbres (*n_estimators* = 200) ce qui améliore la stabilité des prédictions, puis, avec une profondeur plus élevée (*max_depth* = 30) qui permet de capturer davantage de non-linéarités. Les contraintes sur les splits (*min_samples_split* = 4, *min_samples_leaf* = 2) limitent également le surapprentissage tout en conservant une bonne expressivité, et finalement le choix de *max_features* = 'sqrt' qui est cohérent avec les bonnes pratiques pour les forêts aléatoires.

Ces réglages permettent au modèle de mieux exploiter la structure complexe des données, tout en conservant une bonne généralisation temporelle.

L'optimisation par Optuna apporte un gain réel et mesurable au Random Forest :

- amélioration du score moyen,
- meilleure cohérence entre les splits,
- modèle plus performant sur les années récentes,
- capacité accrue à capturer les interactions complexes.

Le Random Forest optimisé devient ainsi l'un des meilleurs modèles du pipeline, surpassant légèrement sa version initiale et confirmant son statut de modèle robuste et performant pour ce type de données.

6.3. Gradient Boosting

```
import optuna

pipeline_gb = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("model", GradientBoostingClassifier())
])

def objective_gb(trial):
    params = {
        "model__n_estimators": trial.suggest_int("n_estimators", 50,
200),
        "model__learning_rate": trial.suggest_float("learning_rate",
0.01, 0.3),
        "model__max_depth": trial.suggest_int("max_depth", 3, 10),
        "model__min_samples_split": trial.suggest_int("min_samples_split", 2, 10),
        "model__min_samples_leaf": trial.suggest_int("min_samples_leaf", 1, 5),
        "model__subsample": trial.suggest_float("subsample", 0.5, 1.0)
    }

    pipeline_gb.set_params(**params)

    # Utilisation de la validation chronologique
    score = validation_chronologique(pipeline_gb, data, annees)

    return score

study_gb = optuna.create_study(direction="maximize")
study_gb.optimize(objective_gb, n_trials=20)

print("Best params GB :", study_gb.best_params)
print("Best score GB :", study_gb.best_value)
```

Pour le modèle Gradient Boosting, plusieurs hyperparamètres vont être testés:

- *n_estimators* : Nombre d'arbres successifs dans le boosting
- *learning_rate* : Vitesse d'apprentissage, plus petit = plus stable mais plus lent.
- *max_depth* : Profondeur maximale des arbres.

- *min_samples_split* : Nombre minimum d'échantillons pour diviser un nœud
- *min_samples_leaf* : Nombre minimum d'échantillons dans une feuille
- *subsample* : Proportion d'échantillons utilisés à chaque itération

L'hyperparamètre au cœur du modèle est *learning_rate*, parce qu'il contrôle la vitesse à laquelle le modèle apprend. Chaque nouvel arbre corrige les erreurs du précédent, mais le *learning_rate* détermine à quel point il corrige. Il contrôle la stabilité, la vitesse d'apprentissage et le risque d'overfitting.

L'optimisation des hyperparamètres avec Optuna a été appliquée au Gradient Boosting en utilisant la validation croisée chronologique, ce qui permet d'évaluer la capacité du modèle à généraliser sur des données futures.

Les meilleurs paramètres trouvés sont :

- *n_estimators* = 130
- *learning_rate* = 0.0812
- *max_depth* = 8
- *min_samples_split* = 5
- *min_samples_leaf* = 1
- *subsample* = 0.8256

Score moyen obtenu : 0.80958

L'optimisation n'a pas pu aller jusqu'au bout des 20 essais prévus. En effet, après 8–9 heures d'exécution, la session Colab s'est interrompue (probablement déconnexion ou dépassement mémoire), ce qui a conduit à 16 trials complétés. Malgré cela, les résultats obtenus sont cohérents et suffisamment stables pour tirer des conclusions fiables.

Pour les scores chronologiques, les scores obtenus sur chaque split (≈ 0.797 à 0.827) sont légèrement supérieurs à ceux obtenus avant optimisation. On peut en conclure que le modèle gagne en expressivité tout en conservant une bonne stabilité. Pour les scores moyens, on peut noter une amélioration notable d'environ $+0.0076$.

- Avant Optuna ≈ 0.8020
- Après Optuna ≈ 0.8096

Pour l'écart-type, avant Optuna ≈ 0.0155 , puis après Optuna ≈ 0.0139 , on y voit une légère diminution, signe d'une meilleure régularité entre les splits.

À l'instar de la régression logistique, mais dans la même continuité que le Random forest, le Gradient Boosting bénéficie réellement de l'optimisation :

- Le nombre d'arbres (*n_estimators* = 130) et la profondeur (*max_depth* = 8) permettent de capturer davantage de non-linéarités.
- Le *learning_rate* ≈ 0.08 offre un bon compromis entre stabilité et capacité d'apprentissage.
- Le *subsample* ≈ 0.83 introduit un léger bagging, ce qui réduit le risque de surapprentissage.
- Les paramètres de split (*min_samples_split* = 5, *min_samples_leaf* = 1) permettent au modèle d'explorer des structures fines tout en restant régulier.

Ces réglages améliorent la capacité du modèle à exploiter la structure complexe des données, tout en conservant une bonne généralisation temporelle. Le fait que l'optimisation n'ait pas pu aller au bout des 20 essais n'est pas problématique, les résultats sont cohérents, le meilleur score est stable et les paramètres trouvés sont plausibles et alignés avec les bonnes pratiques du Gradient Boosting.

Pour conclure l'optimisation par Optuna apporte un gain réel et mesurable au Gradient Boosting, on y voit une amélioration du score moyen, une meilleure stabilité entre les splits, un modèle plus performant sur les années récentes, et une capacité accrue à capturer les interactions complexes. Même avec un nombre de trials incomplet, Optuna a permis d'identifier une configuration plus performante que la version initiale.

Le Gradient Boosting optimisé devient ainsi l'un des meilleurs modèles du pipeline, au même niveau que le Random Forest optimisé.

6.4. Conclusion

Les résultats montrent que chaque modèle bénéficie différemment de l'optimisation : la régression logistique, déjà proche de son optimum, ne progresse que légèrement, tandis que le Random Forest et surtout le Gradient Boosting tirent pleinement parti de l'exploration d'hyperparamètres, avec des gains significatifs en score moyen et en stabilité. Finalement, Optuna confirme non seulement les forces intrinsèques de chaque modèle, mais permet aussi d'identifier des configurations plus robustes et mieux adaptées aux données, faisant du Gradient Boosting et du Random Forest optimisés les approches les plus performantes pour ce problème.

7. Analyse des variables importantes (Feature Importance)

Après avoir évalué les performances globales des modèles et optimisé leurs hyperparamètres à l'aide d'Optuna, il est nécessaire de comprendre comment ces modèles prennent leurs décisions. L'analyse de l'importance des variables constitue une étape essentielle pour interpréter les résultats, valider la cohérence des modèles et identifier les facteurs qui contribuent réellement à la gravité d'un accident.

L'objectif est d'examiner la manière dont chaque modèle exploite les informations disponibles. Cette analyse permet de vérifier si les modèles s'appuient sur des variables pertinentes, mais aussi de repérer d'éventuelles anomalies ou dépendances inattendues. Elle offre également un éclairage utile pour la réduction de dimension, notamment dans le cas de la régression logistique, plus sensible au bruit et à la redondance que les modèles d'arbres.

Pour chacun des trois modèles sélectionnés, deux graphiques sont présentés : d'une part, les 15 variables les plus importantes, qui révèlent les facteurs déterminants dans la prédiction ; d'autre part, les 15 variables les moins importantes, qui permettent d'identifier les modalités peu informatives ou totalement ignorées par les modèles. Ces visualisations facilitent la comparaison entre approches linéaires et modèles d'arbres, et mettent en évidence les différences de comportements entre les algorithmes.

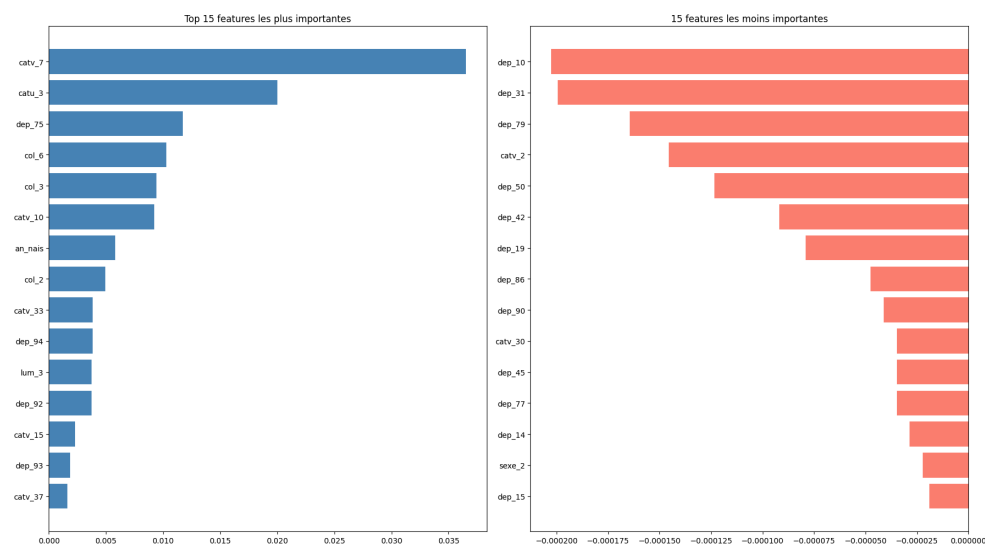
7.1. Logistic Regression

Contrairement aux modèles d'arbres, la régression logistique ne fournit pas directement une mesure d'importance des variables exploitable dans un contexte fortement régularisé et avec un grand nombre de variables catégorielles encodées. Les coefficients bruts ne sont pas interprétables en valeur absolue, et leur magnitude est fortement affectée par la standardisation et la régularisation.

Pour cette raison, l'analyse de l'importance des variables a été réalisée à l'aide de la permutation importance, une méthode agnostique au modèle qui consiste à mesurer la dégradation de performance lorsque l'on perturbe une variable donnée. Concrètement, après avoir entraîné le pipeline optimisé, chaque variable a été permutée aléatoirement dans l'ensemble de test, et la baisse moyenne de performance a été enregistrée. Le calcul a été effectué à l'aide de la fonction *permutation_importance* de scikit-learn, avec trois répétitions pour stabiliser les résultats :

```
result = permutation_importance(  
    pipeline,  
    X_test,  
    y_test,  
    n_repeats=3,  
    random_state=42  
)
```

Cette méthode permet d'obtenir une mesure directe de l'impact de chaque variable sur la capacité du modèle à prédire correctement la gravité d'un accident, indépendamment de l'échelle ou du codage des données.



Feature Importance - Logistic Regression

Les résultats montrent que les variables les plus importantes concernent principalement la nature de la collision, la catégorie de véhicule, la catégorie d'utilisateur, certains départements, l'année de naissance et une modalité particulière de luminosité correspondant à la nuit sans éclairage public. Ces résultats sont cohérents avec les analyses exploratoires : les collisions frontales ou violentes, les usagers

vulnérables (piétons, deux-roues), l'âge, et les conditions de visibilité réduite sont des facteurs bien connus de gravité.

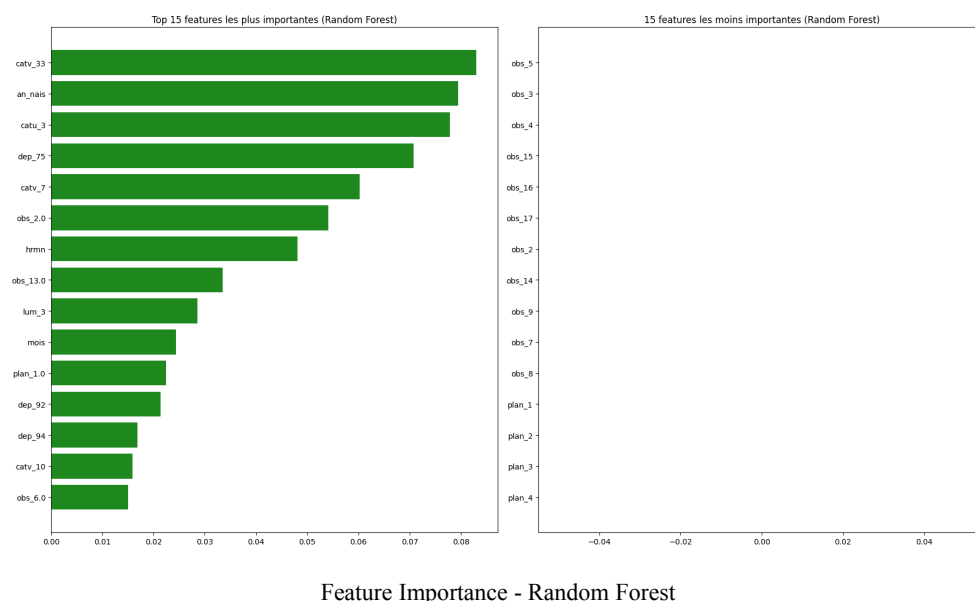
À l'inverse, les variables les moins importantes incluent principalement des départements peu représentés, la modalité correspondant au sexe féminin, ainsi que certaines catégories de véhicules très spécifiques comme les scooters de moins de 50 cm³. Ces variables n'apportent pas d'information discriminante pour la régression logistique, ce qui se reflète dans leur importance proche de zéro.

Une question importante était de savoir si la suppression des variables les moins importantes pouvait améliorer les performances du modèle. Plusieurs tests ont été réalisés en réduisant progressivement le nombre de variables, mais une réduction trop agressive (100 ou 150 variables) a entraîné une baisse notable du score. La régression logistique, fortement régularisée dans notre cas, s'appuie en réalité sur un ensemble plus large de variables pour stabiliser ses coefficients. Même si certaines variables présentent une importance faible ou nulle lorsqu'elles sont perturbées individuellement, elles contribuent indirectement à la stabilité du modèle, notamment en présence de nombreuses variables corrélées.

C'est pour cette raison que le modèle final a été conservé avec environ 200 variables : on obtient une très légère augmentation, avec un score moyen de 0,80203, contre 0,80197 précédemment. Ce compromis permet de réduire le bruit tout en préservant la stabilité et la capacité de généralisation du modèle. La *permutation importance* a donc joué un rôle intéressant, non seulement pour interpréter les résultats, mais aussi pour guider la réduction de dimension de manière raisonnée.

7.2. Random Forest

Pour le Random Forest, l'analyse de l'importance des variables repose directement sur les mesures internes du modèle, qui quantifient la contribution de chaque variable à la réduction d'impureté au sein des arbres. Contrairement à la régression logistique, il n'est pas nécessaire d'utiliser une méthode externe comme la *permutation importance* : le Random Forest fournit une estimation robuste et directement interprétable de l'influence de chaque variable.



Les résultats montrent que les variables les plus importantes incluent plusieurs départements, l'horaire de l'accident, différentes catégories de véhicules, certains obstacles fixes, ainsi qu'une modalité du

tracé de la route correspondant à une portion rectiligne. La présence des départements parmi les variables les plus influentes reflète des différences structurelles entre territoires, déjà observées dans les analyses exploratoires. L'horaire de l'accident apparaît également comme un facteur déterminant, ce qui est cohérent avec les variations de risque selon les périodes de la journée (trafic dense, fatigue, alcool, faible luminosité).

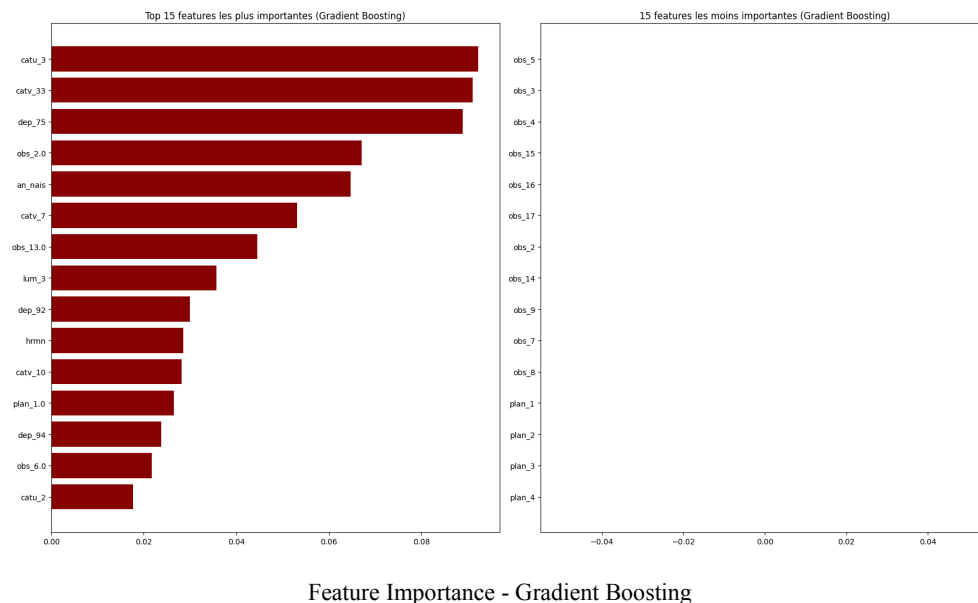
Une variable ressort toutefois de manière plus inattendue : le tracé rectiligne de la route (*plan_1*). Dans nos précédentes analyses descriptives, nous avons constaté que ce type de tracé n'était pas particulièrement associé à une proportion plus élevée d'accidents graves ; il était surtout caractérisé par un nombre total d'accidents plus important. Le Random Forest ne suggère pas pour autant que les routes rectilignes sont intrinsèquement plus dangereuses. Son importance élevée peut s'expliquer par le fait que cette modalité intervient fréquemment dans les données et qu'elle interagit avec d'autres variables (vitesse, type de véhicule, luminosité, obstacles fixes). Le modèle peut donc utiliser cette variable comme un point d'ancrage pour structurer certains splits, non pas parce qu'elle est fortement prédictive seule, mais parce qu'elle contribue à améliorer la séparation des observations lorsqu'elle est combinée à d'autres facteurs. Cette importance doit donc être interprétée comme un effet structurel lié à la fréquence et aux interactions, plutôt que comme un indicateur direct de gravité.

Les variables les moins importantes correspondent principalement à certaines modalités d'obstacles fixes et de tracé de la route. Toutes présentent une importance très proche de zéro, ce qui signifie que le modèle ne les a quasiment jamais utilisées dans un split. Cela explique que le graphique des quinze variables les moins importantes puisse apparaître visuellement vide : les importances sont si faibles qu'aucune barre n'est visible.

Enfin, la question de réentraîner un Random Forest sur un sous-ensemble de variables a été examinée. Contrairement à la régression logistique, les modèles d'arbres ne bénéficient généralement pas d'une réduction manuelle du nombre de variables. Le Random Forest sélectionne déjà automatiquement les variables pertinentes et ignore celles qui ne contribuent pas à la prédiction. Supprimer les variables d'importance nulle ne modifie ni la structure des arbres, ni les performances du modèle, et n'apporte aucun gain de stabilité ou de généralisation. Pour cette raison, le modèle final a été conservé dans sa version complète, sans réduction du nombre de variables.

7.3. Gradient Boosting

L'analyse de l'importance des variables pour le Gradient Boosting repose sur les mesures internes du modèle, qui évaluent la contribution de chaque variable à la réduction de l'erreur au fil des itérations. Comme pour le Random Forest, cette méthode est parfaitement adaptée aux modèles d'arbres, qui sélectionnent naturellement les variables les plus informatives.



Les résultats montrent que les variables les plus importantes incluent deux catégories d'usagers, à savoir les passagers et les piétons, plusieurs départements, différentes catégories de véhicules, certains obstacles fixes, l'année de naissance, l'horaire de l'accident et la modalité de luminosité correspondant à la nuit sans éclairage public. Ces résultats sont cohérents avec les analyses exploratoires : les usagers vulnérables, les conditions de visibilité réduite, l'âge, le type de véhicule et les obstacles fixes sont des facteurs bien connus de gravité.

Une variable apparaît également parmi les plus importantes : le tracé rectiligne de la route (*plan_1*). Comme pour le Random Forest, cette importance doit être interprétée avec prudence.

Les variables les moins importantes correspondent principalement à certaines modalités d'obstacles fixes et de tracé de la route. Comme pour le Random Forest, toutes ces variables présentent une importance très proche zéro, ce qui signifie qu'elles n'ont jamais été utilisées dans un split, ce qui explique que le graphique paraît lui aussi vide. Ce comportement est normal et reflète simplement le fait que le modèle ignore totalement ces variables.

Enfin, concernant la possibilité de réentraîner un Gradient Boosting sur un sous-ensemble de variables, les mêmes arguments que pour le Random Forest s'appliquent. Les modèles d'arbres sélectionnent automatiquement les variables pertinentes et ne tirent aucun bénéfice d'une réduction manuelle du nombre de variables. Supprimer les variables d'importance nulle ne modifie pas les splits déjà optimisés, et n'améliore ni la performance ni la stabilité du modèle. Pour cette raison, le Gradient Boosting final a été conservé dans sa version complète, sans réduction du nombre de variables.

8. Sélection du meilleur modèle

L'ensemble du travail mené avait pour objectif d'identifier le modèle le plus pertinent pour prédire la gravité des accidents de la route. Plusieurs étapes successives ont été nécessaires pour y parvenir. Dans un premier temps, cinq modèles ont été évalués de manière exploratoire : la régression logistique, l'arbre de décision, le Random Forest, le Knn et le Gradient Boosting. Cette première

comparaison a permis d'obtenir une vision d'ensemble de leurs performances et de procéder à une première sélection en choisissant les trois meilleurs.

Ces trois modèles – la régression logistique, le Random Forest et le Gradient Boosting – ont ensuite fait l'objet d'une optimisation approfondie de leurs hyperparamètres à l'aide d'Optuna. Cette étape a permis d'améliorer leurs performances respectives et d'obtenir des versions plus stables et mieux adaptées aux données. Une fois ces modèles optimisés, une analyse détaillée de l'importance des variables a été réalisée. Cette analyse a montré que, malgré des mécanismes internes différents, les modèles d'arbres et la régression logistique s'appuyaient globalement sur un ensemble cohérent de variables pour effectuer leurs prédictions, ce qui renforce la crédibilité des résultats obtenus.

La dernière étape de l'étude a consisté à comparer en détail les performances finales des deux modèles les plus performants, le Random Forest et le Gradient Boosting, en s'appuyant sur leurs matrices de confusion, leurs métriques de classification et leurs validations chronologiques. Cette comparaison est essentielle, car l'objectif du projet n'est pas seulement d'obtenir une bonne performance globale, mais surtout de détecter au mieux les accidents graves, qui constituent la classe minoritaire et la plus critique.

Dans ce contexte, l'accuracy ne suffit pas à elle seule pour départager les modèles. Les deux approches présentent en effet des scores très proches, de l'ordre de 0,83 sur l'ensemble de test, et une stabilité temporelle comparable, avec des scores moyens de validation chronologique de 0,808 pour le Random Forest et de 0,809 pour le Gradient Boosting. Ces résultats montrent que les deux modèles généralisent correctement dans le temps, mais ils ne renseignent pas sur leur capacité à identifier les accidents graves.

L'analyse de la classe 1 révèle en revanche des différences plus marquées. Le Random Forest identifie correctement 4 657 accidents graves sur 19 555, soit un rappel de 0,24 et un f1-score de 0,34 pour cette classe. Le Gradient Boosting détecte 6 125 accidents graves sur le même total, ce qui correspond à un rappel de 0,31 et à un f1-score de 0,40. Il parvient donc à identifier un nombre significativement plus élevé d'accidents graves, au prix d'un léger accroissement du nombre de faux positifs, tout en conservant une précision comparable (0,59 pour le Random Forest contre 0,56 pour le Gradient Boosting). Dans un contexte où la sensibilité à la classe 1 est prioritaire, cette amélioration du rappel et du f1-score est déterminante, même si l'accuracy globale reste similaire.

Par ailleurs, l'analyse de l'importance des variables a montré que les deux modèles s'appuient sur un ensemble de facteurs largement similaires pour effectuer leurs prédictions, ce qui renforce la cohérence des résultats obtenus. Les différences observées ne tiennent donc pas à un désaccord sur les variables pertinentes, mais à la manière dont chaque modèle exploite ces informations pour séparer les observations, en particulier celles correspondant aux accidents graves.

Ainsi, au terme de l'ensemble des analyses menées, évaluation initiale, optimisation des hyperparamètres, interprétation des variables et comparaison finale, le Gradient Boosting apparaît comme le modèle le plus adapté. Il offre le meilleur compromis entre performance globale, stabilité temporelle et capacité à identifier les accidents graves, et constitue donc le modèle retenu pour la suite de l'étude.

9. Création du site web

Nous avons mis en ligne nos modèles au sein d'un site web dédié, nommé *predicgravacc*, accessible à l'adresse suivante : predicgravacc.onrender.com. Ce site constitue l'interface principale permettant de consulter les performances des modèles et d'effectuer des simulations.

9.1. Architecture du site

Le site web est structuré en six pages. La page d'accueil, nommée *index.html* propose un menu de navigation pour se rendre sur les autres pages du site et présente aussi les performances croisées des modèles. Chaque modèle dispose ensuite d'une page dédiée respectant une charte graphique et informative standardisée : contexte, analyse approfondie, métriques de performance et matrice de confusion.

L'aspect interactif est assuré par un module de simulation accessible via un formulaire, ce dernier étant lui-même accessible sur toutes les pages du site. Ce module prend la forme d'un formulaire combinant des menus déroulants et des champs à saisie manuelle. Les valeurs saisissables (comme l'année de naissance, l'heure ou le département) sont encadrées par des contraintes définies en JavaScript, afin de garantir des entrées réalistes et compatibles avec le domaine de définition du modèle. Ce choix de conception permet à la fois de limiter les erreurs de saisie et d'assurer que les données envoyées au modèle restent cohérentes avec celles utilisées lors de l'entraînement.

Une fois le formulaire validé, l'utilisateur est redirigé vers une page dédiée qui affiche clairement la prédiction obtenue à partir des informations fournies (accident léger ou grave). La page présente également un récapitulatif complet des valeurs saisies dans le formulaire, afin que l'utilisateur puisse vérifier les données ayant conduit au résultat.

9.2. Préparation des modèles

Le fichier *train.py* a pour rôle de préparer les données, d'entraîner les modèles et de générer les fichiers nécessaires au fonctionnement du site.

Il commence par charger et harmoniser les données de plusieurs années, en appliquant les mêmes transformations que celles utilisées lors de l'analyse.

Une fois les données consolidées, les trois pipelines sont entraînés :

- Random Forest,
- Régression Logistique,
- Gradient Boosting.

Chaque pipeline inclut les étapes nécessaires (imputation, normalisation éventuelle, modèle) afin de garantir que les transformations appliquées en production soient identiques à celles utilisées lors de l'apprentissage.

Après l'entraînement, les modèles sont exportés au format *.pkl* grâce à *joblib*.

Un fichier *features.pkl* est également généré : il contient la liste complète des colonnes attendues par les modèles, ce qui permet de reconstruire correctement le vecteur d'entrée lors des prédictions.

Ces fichiers *.pkl* constituent la base du backend : ce sont eux que l'application web charge pour effectuer les prédictions en temps réel.

9.3. Flask : moteur applicatif du site

Le cœur applicatif du site web a été développé avec le micro-framework Flask. Ce choix se justifie par sa légèreté et sa capacité à interfacier facilement des scripts de traitement de données et des modèles de Machine Learning avec une interface utilisateur web.

Flask va agir comme un pont entre le moteur de calcul (ici nous avons utilisé Python) et l'interface de consultation (le navigateur). Pour cela, il remplit trois missions principales :

- **La gestion des routes**
Il définit les points d'accès du site (ex: */index*, */random_forest*). Pour chaque URL, Flask va exécuter une fonction spécifique et prédéfinie pour renvoyer le fichier HTML correspondant.
- **Le chargement des modèles**
Au démarrage de l'application, Flask charge la liste des variables attendues par les modèles (*features.pkl*). Les modèles eux-mêmes sont chargés selon un mécanisme de lazy loading : ils ne sont importés en mémoire que lorsqu'ils sont réellement sollicités par l'utilisateur. Cette stratégie permet de réduire la consommation mémoire et d'éviter les erreurs liées au chargement simultané de plusieurs fichiers *.pkl*.
- **Le moteur de templates**
Grâce à l'outil Jinja2, Flask peut injecter des données dynamiques, comme les résultats obtenus suite aux données rentrées dans le formulaire, directement dans les pages HTML.

Concernant la prédiction réalisée grâce aux modèles, elle suit un flux de données rigoureux pour garantir la fiabilité des résultats.

1. **Récupération des données du formulaire**
Lorsque l'utilisateur valide le formulaire, Flask intercepte les variables saisies via la méthode POST.
2. **Préparation des données**
Les entrées textuelles sont transformées en un format numérique que les algorithmes vont pouvoir lire. Pour ça, un DataFrame Pandas est initialisé avec toutes les caractéristiques de notre jeu de données original et les variables sont encodées pour correspondre à celles utilisées pour entraîner le modèle.
3. **Appel du modèle**
Le modèle sélectionné par l'utilisateur reçoit ce vecteur de données et génère une prédiction.
4. **Traduction du résultat**
Flask convertit la sortie du modèle en un label compréhensible et l'injecte dans la page HTML des résultats.

9.3. Render

Enfin pour mettre en ligne notre site web et permettre à tous de s'y rendre, nous avons choisi de la déployer sur la plateforme Render. Il s'agit d'une plateforme Platform-as-a-Service (PaaS) qui permet

d'automatiser la mise en ligne d'applications web en se connectant directement à notre dépôt de code, dans notre cas notre dépôt GitHub.

Le passage d'un environnement de développement local à un serveur de production a nécessité l'ajout de trois fichiers de configuration essentiels pour le fonctionnement du site :

- Le fichier *runtime.txt* précise à Render la version exacte de l'interpréteur Python à utiliser pour créer l'environnement virtuel. Dans notre cas, nous avons fixé la version à 3.11.4 afin de garantir la compatibilité entre nos tests locaux et le serveur distant.
- Le fichier *requirements.txt* liste l'ensemble des dépendances et bibliothèques nécessaires au fonctionnement du projet. Outre le framework Flask (3.1.3) , on y retrouve des outils de calcul scientifique comme NumPy (2.4.4) , Pandas (3.0.2) et Scikit-learn (1.8.0), indispensables au chargement et à l'exécution de nos modèles de Machine Learning.
- *Procfile* est le fichier d'instruction de démarrage du serveur. Il indique à Render d'utiliser Gunicorn , un serveur HTTP robuste pour Python, pour lancer notre application (*app:app*). Contrairement au serveur de test intégré à Flask, Gunicorn est conçu pour gérer plusieurs requêtes simultanées, ce qui est impératif pour une mise en production.

Une autre adaptation a été nécessaire concernant le modèle Random Forest. La version optimisée obtenue via Optuna générait un fichier *.pkl* de plus de 400 Mo, en raison notamment d'un nombre élevé d'arbres et d'une profondeur importante. Ce fichier, trop volumineux pour l'environnement de production de Render, provoquait un dépassement de mémoire lors du chargement du modèle et empêchait l'exécution des simulations. Ce poids important s'explique par la combinaison :

- d'un nombre élevé d'arbres (*n_estimators=200*),
- d'une profondeur maximale élevée (*max_depth=30*),
- et d'un grand nombre de nœuds stockés dans chaque arbre.

Dans un environnement local, ce modèle fonctionnait correctement.

En revanche, sur Render, le chargement d'un fichier aussi lourd provoquait un dépassement de mémoire lors de l'inférence, ce qui entraînait l'interruption du service lors des simulations.

Pour garantir la stabilité du site en production, nous avons donc entraîné une version allégée du Random Forest, en réduisant notamment :

- le nombre d'arbres (*n_estimators=120*),
- la profondeur maximale (*max_depth=15*),
- la taille minimale des feuilles et des splits.

Cette version conserve une structure similaire et des performances satisfaisantes, tout en restant compatible avec les contraintes du serveur.

Grâce à cette configuration, le déploiement est devenu continu : chaque modification apportée au code est automatiquement détectée par Render, qui reconstruit l'environnement et met à jour le site web sans interruption de service. Cette approche garantit que la version en ligne est toujours le reflet exact de nos derniers travaux.

10. Conclusion et perspectives

Dans ce projet, nous nous sommes intéressés à la problématique suivante : *“Comment être capable de prédire de manière fiable la gravité d’un accident de la route à partir de modèles de machine learning optimisés, afin d’améliorer la prévention et l’aide à la décision en matière de sécurité routière ?”*

Pour y répondre, nous avons mis en place une démarche complète allant de la préparation des données à la mise en production d’un site web interactif.

La première étape a consisté à constituer un jeu de données propre, cohérent et représentatif. Les données issues du fichier national d’accidents ont nécessité un travail important de nettoyage, d’harmonisation et de transformation, notamment en raison d’incohérences structurelles entre les années, de valeurs aberrantes (comme celles observées en 2019) et de modalités particulières qu’il a fallu recoder. L’utilisation de plusieurs années (2014–2020, hors 2019) nous a permis de travailler sur un ensemble riche et varié, tout en évaluant la capacité des modèles à généraliser dans le temps.

Nous avons ensuite mené une analyse exploratoire approfondie afin d’identifier les variables les plus pertinentes et de mieux comprendre les mécanismes associés à la gravité des accidents. Cette étape a guidé la sélection des modèles et la construction des pipelines d’apprentissage supervisé. Trois modèles ont été retenus : la Régression Logistique, le Random Forest et le Gradient Boosting. Leur comparaison a mis en évidence des comportements complémentaires : la Régression Logistique offre une base simple et interprétable, le Random Forest présente une bonne robustesse globale, et le Gradient Boosting se distingue par de meilleures performances sur les accidents graves, une classe pourtant minoritaire et difficile à prédire.

Au-delà de l’analyse, un enjeu majeur du projet consistait à rendre ces modèles accessibles via une interface web. Nous avons ainsi développé une application Flask permettant de saisir des données, de reconstruire dynamiquement le vecteur d’entrée attendu par les modèles, et de fournir une prédiction en temps réel. L’intégration des modèles entraînés au format .pkl, la gestion des routes, le moteur de templates et la préparation des données côté serveur ont constitué des éléments centraux de cette architecture. Le déploiement sur Render a permis de rendre l’application accessible en ligne, malgré certaines contraintes techniques, notamment la nécessité d’alléger le modèle Random Forest pour réduire la taille du fichier et éviter les dépassements de mémoire.

Ce projet met en évidence la complémentarité entre les différentes compétences mobilisées : traitement de données, modélisation statistique, optimisation, développement web et déploiement. Il illustre également la manière dont des modèles de machine learning peuvent être intégrés dans une application concrète destinée à la prise de décision.

Cependant, plusieurs pistes d’amélioration peuvent être envisagées. Sur le plan des données, l’intégration de nouvelles variables (conditions météorologiques, infrastructures, densité de trafic) pourrait enrichir les modèles. Une meilleure gestion des données de 2019, via des techniques avancées de détection et de correction d’anomalies, permettrait peut-être de réintégrer cette année. Sur le plan des modèles, des approches plus récentes comme XGBoost, LightGBM ou des réseaux neuronaux pourraient être explorées. Enfin, l’application web pourrait être étendue avec des visualisations interactives supplémentaires ou une optimisation du temps de réponse.

En définitive, ce travail démontre qu’il est possible de construire une chaîne complète allant de la donnée brute à une application web fonctionnelle, tout en répondant de manière satisfaisante à la problématique initiale. Le projet constitue une base solide pour des développements futurs visant à améliorer la précision des prédictions et à renforcer l’utilité opérationnelle de l’outil dans un contexte de sécurité routière.